

Date and Time Series Objects

FRE6871 & FRE7241, Spring 2026

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

January 20, 2026



NYU

**TANDON SCHOOL
OF ENGINEERING**

Date Objects

R has a `Date` class for date objects (but without time).

The function `as.Date()` parses character strings and coerces numeric objects into `Date` objects.

R stores `Date` objects as the number of days since the *epoch* (January 1, 1970).

The function `difftime()` calculates the difference between `Date` objects, and returns a time interval object of class `difftime`.

The `"+"` and `"-"` arithmetic operators and the `"<"` and `">"` logical comparison operators are overloaded to allow these operations directly on `Date` objects.

numeric *year-fraction* dates can be coerced to `Date` objects using the functions `attributes()` and `structure()`.

```
> Sys.Date() ## Get today's date
> as.Date(1e3) ## Coerce numeric into date object
> datetime <- as.Date("2014-07-14") ## "%Y-%m-%d" or "%Y/%m/%d"
> datetime
> class(datetime) ## Date object
> as.Date("07-14-2014", "%m-%d-%Y") ## Specify format
> datetime + 20 ## Add 20 days
> # Extract internal representation to integer
> as.numeric(datetime)
> datep <- as.Date("07/14/2013", "%m/%d/%Y")
> datep
> # Difference between dates
> difftime(datetime, datep, units="weeks")
> weekdays(datetime) ## Get day of the week
> # Coerce numeric into date-times
> datetime <- 0
> attributes(datetime) <- list(class="Date")
> datetime ## "Date" object
> structure(0, class="Date") ## "Date" object
> structure(10000.25, class="Date")
```

POSIXct Date-time Objects

The POSIXct class in R represents *date-time* objects, that can store both the date and time.

The *clock time* is the time (number of hours, minutes and seconds) in the local *time zone*.

The *moment of time* is the *clock time* in the UTC *time zone*.

POSIXct objects are stored as the number of seconds that have elapsed since the *epoch* (January 1, 1970) in the UTC *time zone*.

POSIXct objects are stored as the *moment of time*, but are printed out as the *clock time* in the local *time zone*.

A *clock time* together with a *time zone* uniquely specifies a *moment of time*.

The function `as.POSIXct()` can parse a character string (representing the *clock time*) and a *time zone* into a POSIXct object.

POSIX is an acronym for "Portable Operating System Interface".

```
> datetime <- Sys.time() ## Get today's date and time
> datetime
> class(datetime) ## POSIXct object
> # POSIXct stored as integer moment of time
> as.numeric(datetime)
> # Parse character string "%Y-%m-%d %H:%M:%S" to POSIXct object
> datetime <- as.POSIXct("2014-07-14 13:30:10")
> # Different time zones can have same clock time
> as.POSIXct("2014-07-14 13:30:10", tz="America/New_York")
> as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Format argument allows parsing different date-time string formats
> as.POSIXct("07/14/2014 13:30:10", format="%m/%d/%Y %H:%M:%S",
+           tz="America/New_York")
```

Operations on POSIXct Objects

The "+" and "-" arithmetic operators are overloaded to allow addition and subtraction operations on POSIXct objects.

The "<" and ">" logical comparison operators are also overloaded to allow direct comparisons between POSIXct objects.

Operations on POSIXct objects are equivalent to the same operations on the internal integer representation of POSIXct (number of seconds since the *epoch*).

Subtracting POSIXct objects creates a time interval object of class *difftime*.

The method `seq.POSIXt` creates a vector of POSIXct *date-times*.

```
> # Same moment of time corresponds to different clock times
> timeny <- as.POSIXct("2014-07-14 13:30:10", tz="America/New_York")
> timeldn <- as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Add five hours to POSIXct
> timeny + 5*60*60
> # Subtract POSIXct
> timeny - timeldn
> class(timeny - timeldn)
> # Compare POSIXct
> timeny > timeldn
> # Create vector of POSIXct times during trading hours
> timev <- seq(
+   from=as.POSIXct("2014-07-14 09:30:00", tz="America/New_York"),
+   to=as.POSIXct("2014-07-14 16:00:00", tz="America/New_York"),
+   by="10 min")
> head(timev, 3)
> tail(timev, 3)
```

Moment of Time and Clock Time

`as.POSIXct()` can also coerce integer objects into `POSIXct`, given an origin in time.

The same *moment of time* corresponds to different *clock times* in different *time zones*.

The same *clock times* in different *time zones* correspond to different *moments of time*.

```
> # POSIXct is stored as integer moment of time
> datetimen <- as.numeric(datetime)
> # Same moment of time corresponds to different clock times
> as.POSIXct(datetimen, origin="1970-01-01", tz="America/New_York")
> as.POSIXct(datetimen, origin="1970-01-01", tz="UTC")
> # Same clock time corresponds to different moments of time
> as.POSIXct("2014-07-14 13:30:10", tz="America/New_York") -
+   as.POSIXct("2014-07-14 13:30:10", tz="UTC")
> # Add 20 seconds to POSIXct
> datetime + 20
```

Methods for Manipulating POSIXct Objects

The generic function `format()` formats R objects for printing and display.

The method `format.POSIXct()` parses POSIXct objects into a character string representing the *clock time* in a given *time zone*.

The method `as.POSIXct.Date()` parses Date objects into POSIXct, and assigns to them the *moment of time* corresponding to midnight UTC.

POSIX is an acronym for "Portable Operating System Interface".

```
> datetime ## POSIXct date and time
> # Parse POSIXct to string representing the clock time
> format(datetime)
> class(format(datetime)) ## Character string
> # Get clock times in different time zones
> format(datetime, tz="America/New_York")
> format(datetime, tz="UTC")
> # Format with custom format strings
> format(datetime, "%m/%Y")
> format(datetime, "%m-%d-%Y %H hours")
> # Trunc to hour
> format(datetime, "%m-%d-%Y %H:00:00")
> # Date converted to midnight UTC moment of time
> as.POSIXct(Sys.Date())
> as.POSIXct(as.numeric(as.POSIXct(Sys.Date())) ,
+   origin="1970-01-01",
+   tz="UTC")
```

POSIX1t Date-time Objects

The POSIX1t class in R represents *date-time* objects, that are stored internally as a list.

The function `as.POSIX1t()` can parse a character string (representing the *clock time*) and a *time zone* into a POSIX1t object.

The method `format.POSIX1t()` parses POSIX1t objects into a character string representing the *clock time* in a given *time zone*.

The function `as.POSIX1t()` can also parse a POSIXct object into a POSIX1t object, and `as.POSIXct()` can perform the reverse.

Adding a number to POSIX1t causes implicit coercion to POSIXct.

POSIXct and POSIX1t are two derived classes from the POSIXt class.

The methods `round.POSIXt()` and `trunc.POSIXt()` round and truncate POSIXt objects, and return POSIX1t objects.

```
> # Parse character string "%Y-%m-%d %H:%M:%S" to POSIX1t object
> datetime <- as.POSIX1t("2014-07-14 18:30:10")
> datetime
> class(datetime) ## POSIX1t object
> as.POSIXct(datetime) ## Coerce to POSIXct object
> # Extract internal list representation to vector
> unlist(datetime)
> datetime + 20 ## Add 20 seconds
> class(datetime + 20) ## Implicit coercion to POSIXct
> trunc(datetime, units="hours") ## Truncate to closest hour
> trunc(datetime, units="days") ## Truncate to closest day
> methods(trunc) ## Trunc methods
> trunc.POSIXt
```

Time Zones and Date-time Conversion

date-time objects require a *time zone* to be uniquely specified.

UTC stands for "Universal Time Coordinated", and is synonymous with GMT, but doesn't change with Daylight Saving Time.

EST stands for "Eastern Standard Time", and is UTC - 5 hours.

EDT stands for "Eastern Daylight Time", and is UTC - 4 hours.

The function `Sys.setenv()` can be used to set the default *time zone*, but the environment variable "TZ" must be capitalized.

```
> # Set time-zone to UTC
> Sys.setenv(TZ="UTC")
> Sys.timezone() ## Get time-zone
> Sys.time() ## Today's date and time
> # Set time-zone back to New York
> Sys.setenv(TZ="America/New_York")
> Sys.time() ## Today's date and time
> # Standard Time in effect
> as.POSIXct("2013-03-09 11:00:00", tz="America/New_York")
> # Daylight Savings Time in effect
> as.POSIXct("2013-03-10 11:00:00", tz="America/New_York")
> datetime <- Sys.time() ## Today's date and time
> # Convert to character in different TZ
> format(datetime, tz="America/New_York")
> format(datetime, tz="UTC")
> # Parse back to POSIXct
> as.POSIXct(format(datetime, tz="America/New_York"))
> # Difference between New_York time and UTC
> as.POSIXct(format(Sys.time(), tz="UTC")) -
+   as.POSIXct(format(Sys.time(), tz="America/New_York"))
```


Manipulating Date-time Objects Using *lubridate*

The package *lubridate* contains functions for manipulating POSIXct date-time objects.

The `ymd()`, `dmy()`, etc. functions parse character and numeric *year-fraction* dates into POSIXct objects.

The `mday()`, `month()`, `year()`, etc. accessor functions extract date-time components.

The function `decimal_date()` converts POSIXct objects into numeric *year-fraction* dates.

The function `date_decimal()` converts numeric *year-fraction* dates into POSIXct objects.

```
> library(lubridate) ## Load lubridate
> # Parse strings into date-times
> as.POSIXct("07-14-2014", format="%m-%d-%Y", tz="America/New_York")
> datetime <- lubridate::mdy("07-14-2014", tz="America/New_York")
> datetime
> class(datetime) ## POSIXct object
> lubridate::dmy("14.07.2014", tz="America/New_York")
>
> # Parse numeric into date-times
> as.POSIXct(as.character(14072014), format="%d%m%Y",
+           tz="America/New_York")
> lubridate::dmy(14072014, tz="America/New_York")
>
> # Parse decimal to date-times
> lubridate::decimal_date(datetime)
> lubridate::date_decimal(2014.25, tz="America/New_York")
> date_decimal(decimal_date(datetime), tz="America/New_York")
```

Time Zones Using *lubridate*

The package *lubridate* simplifies *time zone* calculations.

The package *lubridate* uses the *UTC time zone* as default.

The function `with_tz()` creates a date-time object with the same moment of time in a different *time zone*.

The function `force_tz()` creates a date-time object with the same clock time in a different *time zone*.

```
> datetime <- lubridate::ymd_hms(20140714142010,
+                               tz="America/New_York")
> datetime
> # Get same moment of time in "UTC" time zone
> lubridate::with_tz(datetime, "UTC")
> as.POSIXct(format(datetime, tz="UTC"), tz="UTC")
> # Get same clock time in "UTC" time zone
> lubridate::force_tz(datetime, "UTC")
> as.POSIXct(format(datetime, tz="America/New_York"),
+            tz="UTC")
> # Same moment of time
> datetime - with_tz(datetime, "UTC")
> # Different moments of time
> datetime - force_tz(datetime, "UTC")
```

lubridate Time Span Objects

lubridate has two time span classes: durations and periods.

durations specify exact time spans, such as numbers of seconds, hours, days, etc.

The functions `ddays()`, `dyears()`, etc. return duration objects.

periods specify relative time spans that don't have a fixed length, such as months, years, etc.

periods account for variable days in the months, for Daylight Savings Time, and for leap years.

The functions `days()`, `months()`, `years()`, etc. return period objects.

```
> # Daylight Savings Time handling periods vs durations
> datetime <- as.POSIXct("2013-03-09 11:00:00", tz="America/New_York")
> datetime
> datetime + lubridate::ddays(1) ## Add duration
> datetime + lubridate::days(1) ## Add period
>
> leap_year(2012) ## Leap year
> datetime <- lubridate::dmy("01012012", tz="America/New_York")
> datetime
> datetime + lubridate::dyears(1) ## Add duration
> datetime + lubridate::years(1) ## Add period
```

Adding Time Spans to Date-time Objects

periods allow calculating future dates with the same day of the month, or month of the year.

```
> datetime <- lubridate::ymd_hms(20140714142010, tz="America/New_York")
> datetime
> # Add periods to a date-time
> c(datetime + lubridate::seconds(1), datetime + lubridate::minutes(1),
+   datetime + lubridate::days(1), datetime + period(months=1))
>
> # Create vectors of dates
> datetime <- lubridate::ymd(20140714, tz="America/New_York")
> datetime + 0:2 * period(months=1) ## Monthly dates
> datetime + period(months=0:2)
> datetime + 0:2 * period(months=2) ## bi-monthly dates
> datetime + seq(0, 5, by=2) * period(months=1)
> seq(datetime, length=3, by="2 months")
```

End-of-month Dates

Adding monthly periods can create invalid dates.

The operators `%m+%` and `%m-%` add or subtract monthly periods to account for the variable number of days per month.

This allows creating vectors of end-of-month dates.

```
> # Adding monthly periods can create invalid dates
> datetime <- lubridate::ymd(20120131, tz="America/New_York")
> datetime + 0:2 * period(months=1)
> datetime + period(months=1)
> datetime + period(months=2)
> # Create vector of end-of-month dates
> datetime %m-% months(13:1)
```

Package *RQuantLib* Calendar Functions

The package *RQuantLib* is an interface to the *QuantLib* open source C/C++ library for quantitative finance, mostly designed for pricing fixed-income instruments and options.

The *QuantLib* library also contains calendar functions for determining holidays and business days in many different jurisdictions.

```
> library(RQuantLib) ## Load RQuantLib
>
> # Create daily date series of class "Date"
> datev <- Sys.Date() + -5:2
> datev
>
> # Create Boolean vector of business days
> # Use RQuantLib calendar
> isbusday <- RQuantLib::isBusinessDay(
+   calendar="UnitedStates/GovernmentBond", datev)
>
> # Create daily series of business days
> datev[isbusday]
```

Review of Date-time Classes in R

The `Date` class from the base package is suitable for *daily* time series.

The `POSIXct` class from the base package is suitable for *intra-day* time series.

The `yearmon` and `yearqtr` classes from the `zoo` package are suitable for *quarterly* and *monthly* time series.

```
> datetime <- Sys.Date() ## Create date series of class "Date"
> datev <- datetime + 0:365 ## Daily series over one year
> head(datev, 4) ## Print first few dates
> format(head(datev, 4), "%m/%d/%Y") ## Print first few dates
> # Create daily date-time series of class "POSIXct"
> datev <- seq(Sys.time(), by="days", length.out=365)
> head(datev, 4) ## Print first few dates
> format(head(datev, 4), "%m/%d/%Y %H:%M:%S") ## Print first few
> # Create series of monthly dates of class "zoo"
> monthv <- yearmon(2010+0:36/12)
> head(monthv, 4) ## Print first few dates
> # Create series of quarterly dates of class "zoo"
> qrtv <- yearqtr(2010+0:16/4)
> head(qrtv, 4) ## Print first few dates
> # Parse quarterly "zoo" dates to POSIXct
> Sys.setenv(TZ="UTC")
> as.POSIXct(head(qrtv, 4))
```

Time Series Objects of Class *ts*

Time series are data objects that contain a *date-time* index and data associated with it.

The native time series class in R is *ts*.

ts time series are *regular*, i.e. they can only have an equally spaced *date-time* index.

ts time series have a numeric *date-time* index, usually encoded as a *year-fraction*, or some other unit, like number of months, etc.

For example the date "2015-03-31" can be encoded as a *year-fraction* equal to 2015.244.

The *stats* base package contains functions for manipulating time series objects of class *ts*.

The function *ts()* creates a *ts* time series from a numeric vector or matrix, and from the associated *date-time* information (the number of data per time unit: year, month, etc.).

The *frequency* argument is the number of observations per unit of time.

For example, if the *date-time* index is encoded as a *year-fraction*, then *frequency*=12 means 12 monthly data points per year.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Create daily time series ending today
> startd <- decimal_date(Sys.Date()-6)
> endd <- decimal_date(Sys.Date())
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(6)/100))
> tstep <- NROW(datav)/(endd-startd)
> timeser <- ts(data=datav, start=startd, frequency=tstep)
> timeser ## Display time series
> # Display index dates
> as.Date(date_decimal(zoo::coredata(time(timeser))))
> # bi-monthly geometric Brownian motion starting mid-1990
> timeser <- ts(data=exp(cumsum(rnorm(96)/100)),
+               frequency=6, start=1990.5)
```


Manipulating *ts* Time Series

ts time series don't store their *date-time* indices, and instead store only a "tsp" attribute that specifies the index start and end dates and its frequency.

The *date-time* index is calculated as needed from the "tsp" attribute.

The function `time()` extracts the *date-time* index of a *ts* time series object.

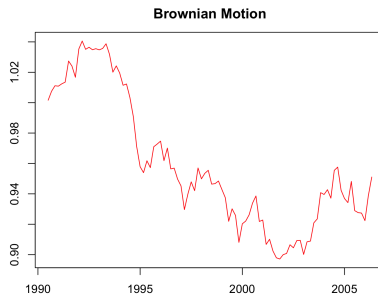
The function `window()` subsets the a *ts* time series object.

```
> # Show some methods for class "ts"
> matrix(methods(class="ts")[3:8], ncol=2)
> # "tsp" attribute specifies the date-time index
> attributes(timeser)
> # Extract the index
> tail(time(timeser), 11)
> # The index is equally spaced
> diff(tail(time(timeser), 11))
> # Subset the time series
> window(timeser, start=1992, end=1992.25)
```

Plotting *ts* Time Series Objects

The method `plot.ts()` plots *ts* time series objects.

```
> # Create plot  
> plot(timeser, type="l", col="red", lty="solid",  
+       xlab="", ylab="")  
> title(main="Brownian Motion", line=1) ## Add title
```



EuStockMarkets Data

R includes a number of base packages that are already installed and loaded.

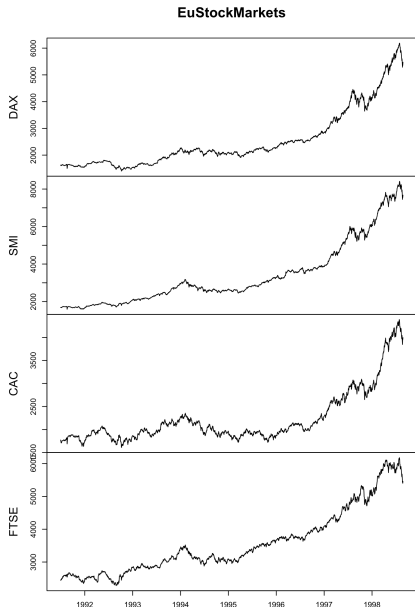
`datasets` is a base package containing various datasets, for example: `EuStockMarkets`.

The `EuStockMarkets` dataset contains daily closing prices of european stock indices.

`EuStockMarkets` is a `mts()` time series object.

The `EuStockMarkets` *date-time* index is equally spaced (*regular*), so the *year-fraction* dates don't correspond to actual trading days.

```
> class(EuStockMarkets) ## Multiple ts object
> dim(EuStockMarkets)
> head(EuStockMarkets, 3) ## Get first three rows
> # EuStockMarkets index is equally spaced
> diff(tail(time(EuStockMarkets), 11))
> # Plot all the columns in separate panels
> plot(EuStockMarkets, main="EuStockMarkets", xlab="")
```

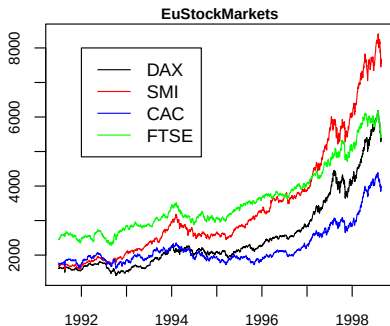


Plotting EuStockMarkets Data

The argument `plot.type="single"` for method `plot.zoo()` allows plotting multiple lines in a single panel (pane).

The four `EuStockMarkets` time series can be plotted in a single panel (pane).

```
> # Plot in single panel
> plot(EuStockMarkets, main="EuStockMarkets",
+      xlab="", ylab="", plot.type="single",
+      col=c("black", "red", "blue", "green"))
> # Add legend
> legend(x=1992, y=8000,
+       legend=colnames(EuStockMarkets),
+       col=c("black", "red", "blue", "green"),
+       lwd=6, lty=1)
```



zoo Time Series Objects

The package *zoo* is designed for managing *irregular* time series and ordered objects of class *zoo*.

Irregular time series have *date-time* indices that aren't equally spaced (because of weekends, overnight hours, etc.).

The function `zoo()` creates a *zoo* object from a numeric vector or matrix, and an associated *date-time* index.

The *zoo* index is a vector of *date-time* objects, and can be from any *date-time* class.

The *zoo* class can manage *irregular* time series whose *date-time* index isn't equally spaced.

```
> library(zoo) ## Load package zoo
> # Create zoo time series of random returns
> datev <- Sys.Date() + 0:11
> zoots <- zoo(rnorm(NROW(datev)), order.by=datev)
> zoots
2026-01-20 2026-01-21 2026-01-22 2026-01-23 2026-01-24 2026-01-25 2026-01-26
0.1450    0.4383    0.1532    1.0849    1.9995    -0.8119
2026-01-27 2026-01-28 2026-01-29 2026-01-30 2026-01-31
0.5859    0.3601   -0.0253    0.1509    0.1101
> attributes(zoots)
$index
[1] "2026-01-20" "2026-01-21" "2026-01-22" "2026-01-23" "2026-01-24"
[6] "2026-01-25" "2026-01-26" "2026-01-27" "2026-01-28" "2026-01-29"
[11] "2026-01-30" "2026-01-31"

$class
[1] "zoo"
> class(zoots) ## Class "zoo"
[1] "zoo"
> tail(zoots, 3) ## Get last few elements
2026-01-29 2026-01-30 2026-01-31
-0.0253    0.1509    0.1101
```

Operations on zoo Time Series

The function `zoo::coredata()` extracts the data contained in `zoo` object, and returns a vector or matrix.

The function `zoo::index()` extracts the time index of a `zoo` object.

The function `xts::.index()` extracts the time index expressed in the number of seconds.

The functions `start()` and `end()` return the time index values of the first and last elements of a `zoo` object.

The functions `cumsum()`, `cummax()`, and `cummin()` return cumulative sums, minima and maxima of a `zoo` object.

```
> zoo::coredata(zoots) ## Extract coredata
> zoo::index(zoots) ## Extract time index
> start(zoots) ## First date
> end(zoots) ## Last date
> zoots[start(zoots)] ## First element
> zoots[end(zoots)] ## Last element
> zoo::coredata(zoots) <- rep(1, NROW(zoots)) ## Replace coredata
> cumsum(zoots) ## Cumulative sum
> cummax(cumsum(zoots))
> cummin(cumsum(zoots))
```

Single Column zoo Time Series

Single column *zoo* time series usually don't have a dimension attribute (they have a NULL dimension), and they don't have a column name, unlike multi-column *zoo* time series.

Single column *zoo* time series without a dimension attribute should be avoided, since they can cause hard to detect bugs.

If a single column *zoo* time series is created from a single column matrices, then it have a dimension attribute, and can be assigned a column name.

```
> zoots <- zoo(matrix(cumsum(rnorm(10))), nc=1,
+   order.by=seq(from=as.Date("2013-06-15"), by="day", len=10))
> colnames(zoots) <- "zoots"
> tail(zoots)
> dim(zoots)
> attributes(zoots)
```

The lag() and diff() Functions

The method `lag.zoo()` returns a lagged version of a *zoo* time series, shifting the time index by "*k*" observations.

If "*k*" is positive, then `lag.zoo()` shifts values from the future to the present, and if "*k*" is negative then it shifts them from the past.

This is the opposite of what is usually considered as a positive *lag*.

A positive *lag* should replace the current value with values from the past (negative lags should replace with values from the future).

The method `diff.zoo()` returns the difference between a *zoo* time series and its proper lagged version from the past, given a positive *lag* value.

By default, the methods `lag.zoo()` and `diff.zoo()` omit any NA values they may have produced, and return shorter time series.

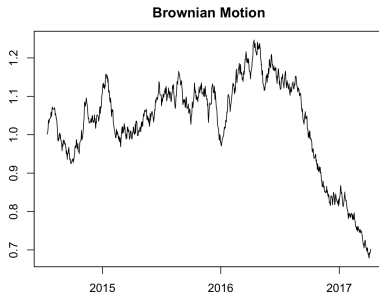
If the "*na.pad*" argument is set to `TRUE`, then they return time series of the same length, with NA values added where needed.

```
> zoo::coredata(zoots) <- (1:10)^2 ## Replace coredata
> zoots
> lag(zoots) ## One day lag
> lag(zoots, 2) ## Two day lag
> lag(zoots, k=-1) ## Proper one day lag
> diff(zoots) ## Diff with one day lag
> # Proper lag and original length
> lag(zoots, -2, na.pad=TRUE)
```


Plotting zoo Time Series

`zoo` time series can be plotted using the generic function `plot()`, which dispatches the `plot.zoo()` method.

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(zoo) ## Load package zoo
> # Create index of daily dates
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=1000)
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(NROW(datev))/100))
> # Create zoo series of geometric Brownian motion
> zoots <- zoo(x=datav, order.by=datev)
> # Plot using method plot.zoo()
> plot.zoo(zoots, xlab="", ylab="")
> title(main="Brownian Motion", line=1) ## Add title
```



Subsetting zoo Time Series

zoo time series can be subset in similar ways to matrices and *ts* time series.

The function `window()` can also subset *zoo* time series objects.

In addition, *zoo* time series can be subset using `Date` objects.

```
> # Subset zoo as matrix
> zoos[459:463, 1]
> # Subset zoo using window()
> window(zoos,
+ start=as.Date("2014-10-15"),
+ end=as.Date("2014-10-19"))
> # Subset zoo using Date object
> zoos[as.Date("2014-10-15")]
```

Sequential Joining zoo Time Series

The zoo time series can be joined sequentially using function `rbind()`.

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(zoo) ## Load package zoo
> # Create daily date series of class "Date"
> tday <- Sys.Date()
> index1 <- seq(tday-2*365, by="days", length.out=365)
> # Create zoo time series of random returns
> zoo1 <- zoo(rnorm(NROW(index1)), order.by=index1)
> # Create another zoo time series of random returns
> index2 <- seq(tday-360, by="days", length.out=365)
> zoo2 <- zoo(rnorm(NROW(index2)), order.by=index2)
> # rbind the two time series - ts1 supersedes ts2
> zoo2b2 <- zoo2[zoo2::index(zoo2) > end(zoo1)]
> zoo3 <- rbind(zoo1, zoo2b2)
> # Plot zoo time series of geometric Brownian motion
> plot(exp(cumsum(zoo3)/100), xlab="", ylab="")
> # Add vertical lines at stitch point
> abline(v=end(zoo1), col="blue", lty="dashed")
> abline(v=start(zoo2), col="red", lty="dashed")
> title(main="Brownian Motions Stitched Together", line=1) ## Add title
```



Merging zoo Time Series

zoo time series can be combined concurrently by joining their columns using function `merge()`.

Function `merge()` is similar to function `cbind()`.

If the `all=TRUE` option is set, then `merge()` returns the union of their dates, otherwise it returns their intersection.

The `merge()` operation can produce NA values.

```
> # Create daily date series of class "Date"
> index1 <- Sys.Date() + -3:1
> # Create zoo time series of random returns
> zoo1 <- zoo(rnorm(NROW(index1)), order.by=index1)
> # Create another zoo time series of random returns
> index2 <- Sys.Date() + -1:3
> zoo2 <- zoo(rnorm(NROW(index2)), order.by=index2)
> merge(zoo1, zoo2) ## union of dates
> # Intersection of dates
> merge(zoo1, zoo2, all=FALSE)
```

Managing NA Values

Binding two time series that don't share the same time index produces NA values.

There are two dedicated functions for managing NA values in time series:

- `stats::na.omit()` removes whole rows of data containing NA values.
- `zoo::na.locf()` replaces NA values with the most recent non-NA values prior to it (*locf* stands for *last observation carry forward*).

Copying the last non-NA values forward causes less data loss than removing whole rows of data.

`na.locf()` with argument `fromLast=TRUE` operates in reverse order, starting from the end.

But copying values forward requires initializing the first row of data, to guarantee that initial NA values are also over-written.

The initial NA *prices* can be initialized to the first non-NA price in the future, which can be done by calling `zoo::na.locf()` with the argument `fromLast=TRUE`.

But the initial NA values in *returns* data should be initialized to *zero*, without carrying data backward from the future, to avoid data *snooping*.

```
> # Create matrix containing NA values
> matv <- sample(18)
> matv[sample(NROW(matv), 4)] <- NA
> matv <- matrix(matv, nc=3)
> # Replace NA values with most recent non-NA values
> zoo::na.locf(matv)
> # Get time series of prices
> pricev <- mget(c("VTI", "VXX"), envir=rutils::etfenv)
> pricev <- lapply(pricev, quantmod::Cl)
> pricev <- rutils::do_call(cbind, pricev)
> sum(is.na(pricev))
> # Carry forward and backward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> pricev <- zoo::na.locf(pricev, na.rm=FALSE, fromLast=TRUE)
> sum(is.na(pricev))
> # Remove whole rows containing NA returns
> retp <- rutils::etfenv$returns
> sum(is.na(retp))
> retp <- na.omit(retp)
> # Or carry forward non-NA returns (preferred)
> retp <- rutils::etfenv$returns
> retp[1, is.na(retp[1, ])] <- 0
> retp <- zoo::na.locf(retp, na.rm=FALSE)
> sum(is.na(retp))
```

Managing NA Values in "xts" Time Series

The function `na.locf.xts()` from package `xts` is faster than `zoo::na.locf()`, but it only operates on time series of class "xts".

```
> # Replace NAs in xts time series
> pricev <- rutils::etfenv$prices[, 1]
> head(pricev)
> sum(is.na(pricev))
> library(quantmod)
> pricezoo <- zoo::na.locf(pricev, na.rm=FALSE, fromLast=TRUE)
> pricexts <- xts::na.locf.xts(pricev, fromLast=TRUE)
> all.equal(pricezoo, pricexts, check.attributes=FALSE)
> library(microbenchmark)
> summary(microbenchmark(
+   zoo=zoo::na.locf(pricev, fromLast=TRUE),
+   xts=xts::na.locf.xts(pricev, fromLast=TRUE),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Coercing Time Series Objects Into zoo

The generic function `as.zoo()` coerces objects into zoo time series.

The function `as.zoo()` creates a zoo object with a numeric *date-time* index, with *date-time* encoded as a *year-fraction*.

The *year-fraction* can be *approximately* converted to a Date object by first calculating the number of days since the *epoch* (1970), and then coercing the numeric days using `as.Date()`.

The function `date_decimal()` from package *lubridate* converts numeric *year-fraction* dates into POSIXct objects.

The function `date_decimal()` provides a more accurate way of converting a *year-fraction* index to POSIXct.

```
> class(EuStockMarkets) ## Multiple ts object
> # Coerce mts object into zoo
> zoos <- as.zoo(EuStockMarkets)
> class(zoo::index(zoos)) ## Index is numeric
> head(zoos, 3)
> # Approximately convert index into class "Date"
> zoo::index(zoos) <- as.Date(365*(zoo::index(zoos)-1970))
> head(zoos, 3)
> # Convert index into class "POSIXct"
> zoos <- as.zoo(EuStockMarkets)
> zoo::index(zoos) <- date_decimal(zoo::index(zoos))
> head(zoos, 3)
```

Coercing zoo Time Series Into Class *ts*

The generic function `as.ts()` from package *stats* coerces time series objects (including *zoo*) into *ts* time series.

The function `as.ts()` creates a *ts* object with a `frequency=1`, implying a “*day*” time unit, instead of a “*year*” time unit suitable for *year-fraction* dates.

A *ts* time series can be created from a *zoo* using the function `ts()`, after extracting the data and date attributes from *zoo*.

The function `decimal_date()` from package *lubridate* converts POSIXct objects into numeric *year-fraction* dates.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Create index of daily dates
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=100)
> # Create vector of geometric Brownian motion
> datav <- exp(cumsum(rnorm(NROW(datev))/100))
> # Create zoo time series of geometric Brownian motion
> zoots <- zoo(x=datav, order.by=datev)
> head(zoots, 3) ## zoo object
> # as.ts() creates ts object with frequency=1
> timeser <- as.ts(zoots)
> tsp(timeser) ## Frequency=1
> # Get start and end dates of zoots
> startd <- decimal_date(start(zoots))
> endd <- decimal_date(end(zoots))
> # Calculate frequency of zoots
> tstep <- NROW(zoots)/(endd-startd)
> datav <- zoo::coredata(zoots) ## Extract data from zoots
> # Create ts object using ts()
> timeser <- ts(data=datav, start=startd, frequency=tstep)
> # Display start of time series
> window(timeser, start=start(timeser), end=start(timeser)+4/365)
> head(time(timeser)) ## Display index dates
> head(as.Date(date_decimal(zoo::coredata(time(timeser)))))
```


Coercing Irregular Time Series Into Class *ts*

Irregular time series cannot be properly coerced into *ts* time series without modifying their index.

The function `as.ts()` creates NA values when it coerces irregular time series into a *ts* time series.

```
> # Create weekday Boolean vector
> wkdays <- weekdays(zoo::index(zoots))
> wkday1 <- !((wkdays == "Saturday") | (wkdays == "Sunday"))
> # Remove weekends from zoo time series
> zoots <- zoots[wkday1, ]
> head(zoots, 7) ## zoo object
> # as.ts() creates NA values
> timeser <- as.ts(zoots)
> head(timeser, 7)
> # Create vector of regular dates, including weekends
> datev <- seq(from=start(zoots), by="day", length.out=NROW(zoots))
> zoo::index(zoots) <- datev
> timeser <- as.ts(zoots)
> head(timeser, 7)
```

Class *xts* Time Series Objects

The package *xts* defines time series objects of class *xts*,

- Class *xts* is an extension of the *zoo* class (derived from *zoo*),
- Class *xts* is the most widely accepted time series class,
- Class *xts* is designed for high-frequency and *OHLC* data,
- Class *xts* contains many convenient functions for plotting, calculating rolling max, min, etc.

The function `xts()` creates a *xts* object from a numeric vector or matrix, and an associated *date-time* index.

The *xts* index is a vector of *date-time* objects, and can be from any *date-time* class.

The *xts* class can manage *irregular* time series whose *date-time* index isn't equally spaced.

```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> library(xts) ## Load package xts
> # Create xts time series of random returns
> datev <- Sys.Date() + 0:3
> xtsv <- xts(rnorm(NROW(datev)), order.by=datev)
> names(xtsv) <- "random"
> xtsv
> tail(xtsv, 3) ## Get last few elements
> first(xtsv) ## Get first element
> last(xtsv) ## Get last element
> class(xtsv) ## Class "xts"
> attributes(xtsv)
> # Get the time zone of an xts object
> tzzone(xtsv)
```

Coercing *zoo* Time Series Into Class *xts*

The function `as.xts()` coerces *zoo* time series into *xts* series.

`as.xts()` preserves the *index* attributes of the original time series.

xts can be plotted using the generic function `plot()`, which dispatches the `plot.xts()` method.

```
> load(file="/Users/jerzy/Develop/lecture_slides/data/zoo_data.RData")
> class(zoo_stx)
> # as.xts() coerces zoo series into xts series
> library(xts) ## Load package xts
> pricexts <- as.xts(zoo_stx)
> dim(pricexts)
> head(pricexts[, 1:4], 4)
> # Plot using plot.xts method
> xts::plot.xts(pricexts[, "Close"], xlab="", ylab="", main="")
> title(main="Stock Prices") ## Add title
```

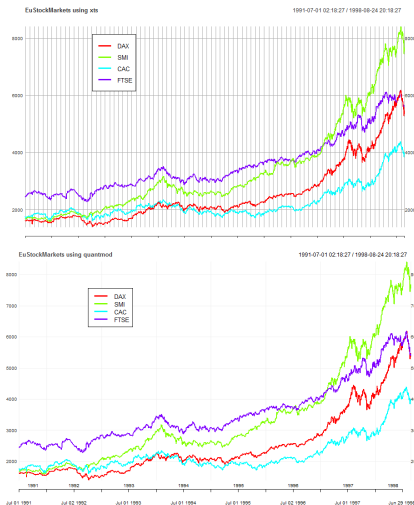


Plotting Multiple xts Using Packages xts and quantmod

```

> library(lubridate) ## Load lubridate
> # Coerce EuStockMarkets into class xts
> xtsv <- xts(zoo::coredata(EuStockMarkets),
+   order.by=date_decimal(zoo::index(EuStockMarkets)))
> # Plot all columns in single panel: xts v.0.9-8
> colorv <- rainbow(NCOL(xtsv))
> plot(xtsv, main="EuStockMarkets using xts",
+   col=colorv, major.ticks="years",
+   minor.ticks=FALSE)
> legend("topleft", legend=colnames(EuStockMarkets),
+   inset=0.2, cex=0.7, , lty=rep(1, NCOL(xtsv)),
+   lwd=3, col=colorv, bg="white")
> # Plot only first column: xts v.0.9-7
> plot(xtsv[, 1], main="EuStockMarkets using xts",
+   col=colorv[1], major.ticks="years",
+   minor.ticks=FALSE)
> # Plot remaining columns
> for (colnum in 2:NCOL(xtsv))
+   lines(xtsv[, colnum], col=colorv[colnum])
> # Plot using quantmod
> library(quantmod)
> plotheme <- chart_theme()
> plotheme$col$line.col <- colors
> chart_Series(x=xtsv, theme=plotheme,
+   name="EuStockMarkets using quantmod")
> legend("topleft", legend=colnames(EuStockMarkets),
+   inset=0.2, cex=0.7, , lty=rep(1, NCOL(xtsv)),
+   lwd=3, col=colorv, bg="white")

```



Plotting xts Using Package ggplot2

xts time series can be plotted using the package *ggplot2*.

The function `qplot()` is the simplest function in the *ggplot2* package, and allows creating line and bar plots.

The function `theme()` customizes plot objects.

```
> library(ggplot2)
> pricev <- rutils::etfenv$prices[, 1]
> pricev <- na.omit(pricev)
> # Create ggplot object
> plotobj <- qplot(x=zoo::index(pricev),
+                 y=as.numeric(pricev),
+                 geom="line",
+                 main=names(pricev)) +
+   xlab("") + ylab("") +
+   theme( ## Add legend and title
+     legend.position=c(0.1, 0.5),
+     plot.title=element_text(vjust=-2.0),
+     plot.background=element_blank()
+   ) ## end theme
> # Render ggplot object
> plotobj
```

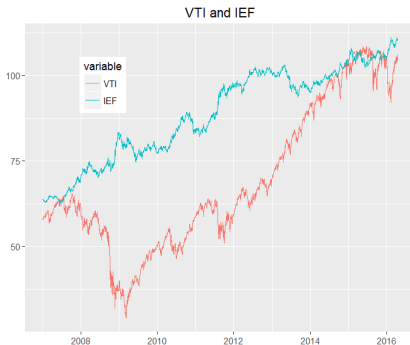


Plotting Multiple xts Using Package *ggplot2*

Multiple xts time series can be plotted using the function `ggplot()` from package *ggplot2*.

But *ggplot2* functions don't accept time series objects, so time series must be first coerced into data frames.

```
> library(rutils) ## Load xts time series data
> library(reshape2)
> library(ggplot2)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Create data frame of time series
> dframe <- data.frame(datev=zoo::index(pricev), zoo::coredata(pricev))
> # reshape data into a single column
> dframe <- reshape2::melt(dframe, id="datev")
> x11(width=6, height=5) ## Open plot window
> # ggplot the melted dframe
> ggplot(data=dframe,
+ mapping=aes(x=datev, y=value, colour=variable)) +
+ geom_line() +
+ xlab("") + ylab("") +
+ ggtitle("VTI and IEF") +
+ theme( ## Add legend and title
+   legend.position=c(0.2, 0.8),
+   plot.title=element_text(vjust=-2.0)
+ ) ## end theme
```



Time series with multiple columns must be reshaped into a single column, which can be performed using the function `melt()` from package *reshape2*,

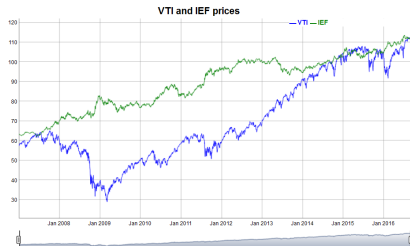
Interactive Time Series Plots Using Package *dygraphs*

The function `dygraph()` from package *dygraphs* creates interactive, zoomable plots from *xts* time series.

The function `dyOptions()` adds options (like colors, etc.) to a *dygraph* plot.

The function `dyRangeSelector()` adds a date range selector to the bottom of a *dygraphs* plot.

```
> # Load rutils which contains etfenv dataset
> library(rutils)
> library(dygraphs)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Plot dygraph with date range selector
> dygraph(pricev, main="VTI and IEF prices") %>%
+   dyOptions(colors=c("blue","green")) %>%
+   dyRangeSelector()
```



The *dygraphs* package in R is an interface to the *dygraphs* JavaScript charting library.

Interactive *dygraphs* plots require running *JavaScript* code, which can be embedded in *html* documents, and displayed by web browsers.

But *pdf* documents can't run *JavaScript* code, so they can't display interactive *dygraphs* plots,

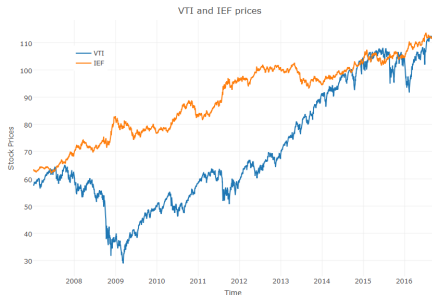
Interactive Time Series Plots Using Package *plotly*

The function `plot_ly()` from package *plotly* creates interactive plots from data residing in data frames.

The function `add_trace()` adds elements to a *plotly* plot.

The function `layout()` modifies the layout of a *plotly* plot.

```
> # Load rutils which contains etfenv dataset
> library(rutils)
> library(plotly)
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> # Create data frame of time series
> dframe <- data.frame(datev=zoo::index(pricev),
+   zoo::coredata(pricev))
> # Plotly syntax using pipes
> dframe %>%
+   plot_ly(x=datev, y=VTI, type="scatter", mode="lines", name="VTI") %>%
+   add_trace(x=datev, y=IEF, type="scatter", mode="lines", name="IEF") %>%
+   layout(title="VTI and IEF prices",
+     xaxis=list(title="Time"),
+     yaxis=list(title="Stock Prices"),
+     legend=list(x=0.1, y=0.9))
> # Or use standard plotly syntax
> plotobj <- plot_ly(data=dframe, x=datev, y=VTI, type="scatter", mode="lines", name="VTI")
> plotobj <- add_trace(p=plotobj, x=datev, y=IEF, type="scatter", mode="lines", name="IEF")
> plotobj <- layout(p=plotobj, title="VTI and IEF prices", xaxis=list(title="Time"), yaxis=list(title="Stock Prices"), legend=list(x=0.1, y=0.9))
> plotobj
```



Subsetting *xts* Time Series

xts time series can be subset in similar ways as *zoo* time series.

In addition, *xts* time series can be subset using date strings, or date range strings, for example: ["2014-10-15/2015-01-10"].

xts time series can be subset by year, week, days, or even seconds.

If only the date is subset, then a comma "," after the date range isn't necessary.

The function `.subset_xts()` allows fast subsetting of *xts* time series, which for large datasets can be faster than the bracket "[" notation.

```
> # Subset xts using a date range string
> pricev <- rutils::etfenv$prices
> pricesub <- pricev["2014-10-15/2015-01-10", 1:4]
> first(pricesub)
> last(pricesub)
> # Subset Nov 2014 using a date string
> pricesub <- pricev["2014-11", 1:4]
> first(pricesub)
> last(pricesub)
> # Subset all data after Nov 2014
> pricesub <- pricev["2014-11/", 1:4]
> first(pricesub)
> last(pricesub)
> # Comma after date range not necessary
> all.equal(pricev["2014-11", ], pricev["2014-11"])
> # .subset_xts() is faster than the bracket []
> library(microbenchmark)
> summary(microbenchmark(
+   bracket=pricev[10:20, ],
+   subset=xts::.subset_xts(pricev, 10:20),
+   times=10))[, c(1, 4, 5)]
```

Fast Subsetting of *xts* Time Series

Subsetting of *xts* time series can be made much faster if the right operations are used.

Subsetting *xts* time series using Boolean vectors is usually faster than using date strings.

But the speed of subsetting can be reduced by additional operations, like coercing strings into dates.

```
> # Specify string representing a date
> datev <- "2014-10-15"
> # Subset prices in two different ways
> pricev <- rutils::etfenv$prices
> all.equal(pricev[zoo::index(pricev) >= datev],
+   pricev[paste0(datev, "/")])
> # Boolean subsetting is slower because coercing string into date
> library(microbenchmark)
> summary(microbenchmark(
+   boolean=(pricev[zoo::index(pricev) >= datev]),
+   date=(pricev[paste0(datev, "/")]),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
> # Coerce string into a date
> datev <- as.Date("2014-10-15")
> # Boolean subsetting is faster than using date string
> summary(microbenchmark(
+   boolean=(pricev[zoo::index(pricev) >= datev]),
+   date=(pricev[paste0(datev, "/")]),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
```

Subsetting Recurring xts Time Intervals

A *recurring time interval* is the same time interval every day, for example the time interval from 9:30AM to 4:00PM every day.

xts series can be subset on recurring time intervals using the "T" notation.

For example, to subset the time interval from 9:30AM to 4:00PM every day: ["T09:30:00/T16:00:00"]

Warning messages that "timezone of object is different than current timezone" can be suppressed by calling the function `options()` with argument `"xts_check_tz=FALSE"`

```
> pricev <- HighFreq::SPY["2012-04"]
> # Subset recurring time interval using "T notation",
> pricev <- pricev["T10:30:00/T15:00:00"]
> first(pricev["2012-04-16"]) ## First element of day
> last(pricev["2012-04-16"]) ## Last element of day
> # Suppress timezone warning messages
> options(xts_check_tz=FALSE)
```

Binding *xts* Time Series by Rows

The function `rbind()` joins the rows of *xts* time series.

If the time series have overlapping time indices then the join produces duplicate rows with the same dates.

The duplicate rows can be removed using the function `duplicated()`.

The function `duplicated()` returns a Boolean vector indicating the duplicate elements of a vector.

The function `duplicated()` with argument `"fromLast=TRUE"` identifies duplicate elements starting from the end.

```
> # Create time series with overlapping time indices
> vti1 <- rutils::etfenv$VTI["/2015"]
> vti2 <- rutils::etfenv$VTI["2014/"]
> dates1 <- zoo::index(vti1)
> dates2 <- zoo::index(vti2)
> # Join by rows
> vti <- rbind(vti1, vti2)
> datev <- zoo::index(vti)
> sum(duplicated(datev))
> vti <- vti[!duplicated(datev), ]
> all.equal(vti, rutils::etfenv$VTI)
> # Alternative method - slightly slower
> vti <- rbind(vti1, vti2[!(zoo::index(vti2) %in% zoo::index(vti1))])
> all.equal(vti, rutils::etfenv$VTI)
> # Remove duplicates starting from the end
> vti <- rbind(vti1, vti2)
> vti <- vti[!duplicated(datev), ]
> vti1f <- vti[!duplicated(datev, fromLast=TRUE), ]
> all.equal(vti, vti1f)
```

Properties of *xts* Time Series

xts series always have a `dim` attribute, unlike *zoo*, which have no `dim` attribute when they only have one column of data.

zoo series with multiple columns have a `dim` attribute, and are therefore matrices.

But *zoo* with a single column don't, and are therefore vectors not matrices.

When a *zoo* is subset to a single column, the `dim` attribute is dropped, which can create errors.

```
> pricev <- rutils::etfenv$prices[, c("VTI", "IEF")]
> pricev <- na.omit(pricev)
> str(pricev) ## Display structure of xts
> # Subsetting zoo to single column drops dim attribute
> pricezoo <- as.zoo(pricev)
> dim(pricezoo)
> dim(pricezoo[, 1])
> # zoo with single column are vectors not matrices
> c(is.matrix(pricezoo), is.matrix(pricezoo[, 1]))
> # xts always have a dim attribute
> rbind(base=dim(pricev), subs=dim(pricev[, 1]))
> c(is.matrix(pricev), is.matrix(pricev[, 1]))
```

lag() and diff() Operations on *xts* Time Series

The methods `xts::lag()` and `xts::diff()` for *xts* series differ from those of package *zoo*.

By default, the method `xts::lag()` replaces the current value with values from the past (negative lags replace with values from the future).

The methods `zoo::lag()` and `zoo::diff()` shorten the series by the number of lag periods.

By default, the methods `xts::lag()` and `xts::diff()` retain the same number of elements, by padding with leading or trailing NA values.

In order to avoid padding with NA values, asset returns can be padded with zeros, and prices can be padded with the first or last elements of the input vector.

```
> # Lag of zoo shortens it by one row
> rbind(base=dim(pricexoo), lag=dim(lag(pricexoo)))
> # Lag of xts doesn't shorten it
> rbind(base=dim(pricex), lag=dim(lag(pricex)))
> # Lag of zoo is in opposite direction from xts
> head(lag(pricexoo, -1), 4)
> head(lag(pricex), 4)
```

Determining Calendar *End points* of *xts* Time Series

The function `endpoints()` from package *xts* extracts the indices of the last observations in each calendar period of time of an *xts* series.

For example:

```
endpoints(x, on="hours")
```

extracts the indices of the last observations in each hour.

The *end points* calculated by `endpoints()` aren't always equally spaced, and aren't the same as those calculated from fixed intervals.

For example, the last observations in each day aren't equally spaced due to weekends and holidays.

```
> # Indices of last observations in each hour
> endd <- xts::endpoints(pricev, on="hours")
> head(endd)
> # Extract the last observations in each hour
> head(pricev[endd, ])
```

Converting xts Time Series to Lower Periodicity

The function `to.period()` converts a time series to a lower periodicity (for example from hourly to daily periodicity).

`to.period()` returns a time series of open, high, low, and close values (*OHLC*) for the lower period.

`to.period()` converts both univariate and *OHLC* time series to a lower periodicity.

```
> # Lower the periodicity to months
> pricem <- to.period(x=pricev, period="months", name="MSFT")
> # Convert colnames to standard OHLC format
> colnames(pricem)
> colnames(pricem) <- sapply(
+   strsplit(colnames(pricem), split=".", fixed=TRUE),
+   function(namev) namev[-1]
+ ) ## end sapply
> head(pricem, 3)
> # Lower the periodicity to years
> pricey <- to.period(x=pricem, period="years", name="MSFT")
> colnames(pricey) <- sapply(
+   strsplit(colnames(pricey), split=".", fixed=TRUE),
+   function(namev) namev[-1]
+ ) ## end sapply
> head(pricey)
```


Plotting OHLC Time Series Using chart_Series()

The function `chart_Series()` from package *quantmod* can plot candlestick plots of OHLC prices.

Each *candlestick* displays one period of data, and consists of a box representing the *Open* and *Close* prices, and a vertical line representing the *High* and *Low* prices.

The color of the box signifies whether the *Close* price was higher or lower than the *Open*,

```
> load(file="/Users/jerzy/Develop/lecture_slides/data/zoo_data.RData")
> library(quantmod) ## Load package quantmod
> # as.xts() coerces zoo series into xts series
> class(zoo_stx)
> pricexts <- as.xts(zoo_stx)
> dim(pricexts)
> head(pricexts[, 1:4], 4)
> # OHLC candlechart
> plotheme <- chart_theme()
> plotheme$col$up.col <- c("green")
> plotheme$col$dn.col <- c("red")
> chart_Series(x=pricexts["2016-05/2016-06", 1:4], theme=plotheme,
+   name="Candlestick Plot of OHLC Stock Prices")
```



Plotting *OHLC* Time Series Using Package *dygraphs*

The function `dygraph()` from package *dygraphs* creates interactive plots for *xts* time series.

The function `dyCandlestick()` creates a *candlestick* plot object for *OHLC* data, and uses the first four columns to plot *candlesticks*, and it plots any additional columns as lines.

The function `dyOptions()` adds options (like colors, etc.) to a *dygraph* plot.

```
> library(dygraphs)
> # Create dygraphs object
> dyplot <- dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4])
> # Convert dygraphs object to candlestick plot
> dyplot <- dygraphs::dyCandlestick(dyplot)
> # Render candlestick plot
> dyplot
> # Candlestick plot using pipes syntax
> dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4]) %>%
+   dyCandlestick() %>%
+   dyOptions(colors="red", strokeWidth=3)
> # Candlestick plot without using pipes syntax
> dygraphs::dyCandlestick(dygraphs::dyOptions(
+   dygraphs::dygraph(pricexts["2016-05/2016-06", 1:4]),
+   colors="red", strokeWidth=3))
```



Each *candlestick* displays one period of data, and consists of a box representing the *Open* and *Close* prices, and a vertical line representing the *High* and *Low* prices.

The color of the box signifies whether the *Close* price was higher or lower than the *Open*,

Time Series Classes in R

R and other packages contain a number of different time series classes:

- Class *ts* from base package *stats*: native time series class in R, but allows only *regular* (equally spaced) date-time index, not suitable for sophisticated financial applications,
- Class *zoo*: allows *irregular* date-time index, the *zoo* index can be from any *date-time* class,
- Class *xts* extension of *zoo* class: most widely accepted time series class, designed for high-frequency and *OHLC* data, contains convenient functions for plotting, calculating rolling max, min, etc.
- Class *timeSeries* from the *Rmetrics* suite,

```
> # Create zoo time series
> datev <- seq(from=as.Date("2014-07-14"), by="day", length.out=10)
> timeser <- zoo(x=sample(10), order.by=datev)
> class(timeser)
> timeser
> library(xts)
> # Coerce zoo time series to class xts
> pricexts <- as.xts(timeser)
> class(xtseries)
> xtseries
```